CODE C TECHNOLOGIES:

2. Automated Resume Parser

Description: Create a system to extract and categorize information from resumes (PDFs/Docs).

Technologies: Python (spaCy, PDFPlumber), Flask, PostgreSQL

Outcome: Extract candidate details (name, skills, education) and store them in a searchable database.

CODE:

```
!pip install flask spacy pdfplumber

!python -m spacy download en_core_web_sm


from flask import Flask, request, redirect, render_template_string

import pdfplumber

import spacy

import sqlite3

# Load NLP model

nlp = spacy.load("en_core_web_sm")

app = Flask(__name__)

# DATABASE (SQLite)

conn = sqlite3.connect("resume.db", check_same_thread=False)

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS candidates (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT,

    skills TEXT,

    education TEXT

)

""")
```

```
def save_candidate(name, skills, education):
    cursor.execute(
        "INSERT INTO candidates (name, skills, education) VALUES (?, ?, ?)",
        (name, skills, education)
    )
    conn.commit()
# PDF TEXT EXTRACTION
def extract_text(file):
    text = ""
    with pdfplumber.open(file) as pdf:
        for page in pdf.pages:
            content = page.extract_text()
            if content:
                text += content + "\n"
    return text
# NLP FUNCTIONS
# Extract Name
def extract_name(text):
    doc = nlp(text)
    for ent in doc.ents:
        if ent.label_ == "PERSON":
            return ent.text
    return "Not Found"


# Skills database
SKILLS = [
    "python", "java", "sql", "machine learning", "data analysis",
    "excel", "power bi", "statistics", "deep learning",
    "flask", "html", "css", "javascript"
]
def extract_skills(text):
```

```
    text = text.lower()
    found = []
    for skill in SKILLS:
        if skill in text:
            found.append(skill)
    return ", ".join(found) if found else "Not Found"

# Extract Education
def extract_education(text):
    keywords = ["bsc", "msc", "btech", "mtech", "phd", "degree", "be", "me"]
    text = text.lower()
    found = []
    for word in keywords:
        if word in text:
            found.append(word.upper())
    return ", ".join(found) if found else "Not Found"
# HTML FRONTEND
HTML_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Resume Parser</title>
    <style>
        body { font-family: Arial; text-align: center; }
        .card {
            border: 1px solid #ccc;
            padding: 10px;
            margin: 10px auto;
            width: 60%;
            border-radius: 10px;
        }
```

```
    </style>
</head>
<body>

<h1> 📄 Automated Resume Parser</h1>

<form action="/upload" method="POST" enctype="multipart/form-data">
    <input type="file" name="resume" required>
    <button type="submit">Upload Resume</button>
</form>

<h2> 📋 Parsed Candidates</h2>

{% for c in candidates %}
<div class="card">
    <p><b>Name:</b> {{c[1]}}</p>
    <p><b>Skills:</b> {{c[2]}}</p>
    <p><b>Education:</b> {{c[3]}}</p>
</div>
{% endfor %}

</body>
</html>
"""
@app.route("/")
def home():
    cursor.execute("SELECT * FROM candidates")
    data = cursor.fetchall()
    return render_template_string(HTML_PAGE, candidates=data)

@app.route("/upload", methods=["POST"])
```

```
def upload():
    file = request.files["resume"]

    if file:
        text = extract_text(file)

        name = extract_name(text)
        skills = extract_skills(text)
        education = extract_education(text)

        save_candidate(name, skills, education)

    return redirect("/")
# RUN APP
if __name__ == "__main__":
    app.run(debug=True)
```

OUTPUT:

Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pdfplumber'
Run completed in 5860.100000000093ms